Contents

TKS v8.0: A Typed Finite-Function Kernel for Cognitive-Architecture Semantics **1**
Abstract . . . 1
1. Motivation and problem statement . . . 2
2. The typed finite-function category . . . 2
2.1 Objects and arrows . . . 2
2.2 Identity and composition . . . 2
2.3 Endomaps versus semigroupoid paths . . . 3
3. The deterministic standard-basis lift . . . 3
3.1 Exact operator-norm theorem . . . 3
3.2 Functoriality . . . 4
4. Stable denotational normal forms . . . 4
5. Formal verification status . . . 4
6. Proof and integration ledger . . . 5
7. Relation to the wider TKS/CAS program . . . 5
8. What TKS v8.0 does not claim . . . 6
9. Reproduction . . . 6
10. Conclusion . . . 6
References . . . 6

TKS v8.0: A Typed Finite-Function Kernel for Cognitive-Architecture Semantics

White paper, version 0.1 — 2026-08-07

Artifact status: `DRAFT / GATE 1 PROTOTYPE / Ring 4 / NOT LICENSED / NOT FROZEN`

Source basis: - `TKS_V8_CORE_FORMAL_SPEC_AND_PROOF_OBLIGATIONS_v0.1.md` (SHA-256 `F740BA05AB7ADB2DCDF55E271016D127E45073B015497DE8982D44A49F0EA2C4`) - `TKS_V8_CORE_PROTOTYPE_MANIFEST_` (artifact `TKS-V8-TYPED-CORE-PROTOTYPE` v0.1.0) - Implementation: `src/tks_v8_core/` (`category.py`, `deterministic_lift.py`, `normal_forms.py`) - Verifier: `scripts/verify_tks_v8_gate1.py`

Authorship / review: `OPEN / NO SUBMISSION DECISION RECORDED / EXTERNAL REVIEW OPEN`

Abstract

TKS v8.0 introduces a deliberately small, executable reference kernel for the typed-composition layer of the Tootra Knowledge System (TKS) and its secular derivative, the Cognitive Architecture System (CAS). The kernel supplies five formal capabilities: (1) named finite objects with explicitly ordered carriers; (2) total finite-function arrows with declared source and target and partial, type-checked categorical composition; (3) a hard separation between one-carrier endomaps, which admit monoid powers, and heterogeneous semigroupoid paths, which do not; (4) a deterministic standard-basis lift into finite-dimensional linear operators with an exact operator-norm theorem; and (5) stable denotational normal forms defined by declared generator order and shortlex minimization over a completed finite closure.

The kernel is an independent reference model. It does not modify, replace, or certify the active CAS notation compiler; it maps no canonical Noetic or CAS construct onto kernel arrows; and it makes no empirical cognitive claim. Its evidentiary status is precisely bounded: pointwise proofs and bounded exhaustive executable tests (19,683 associativity triples, 289 endomaps through carrier size four, 1,678 lift-functor composition pairs), plus a pinned Lean 4.32.1 formal seed that kernel-checks a named subset of the results under a single standard axiom dependency. Universal formalizations, semantic transfer to CAS, compiler integration, and independent specialist review remain open and are enumerated as explicit proof obligations.

---

1. Motivation and problem statement

Prior TKS/CAS formal artifacts (through v7.4 and the v7.5–v7.7 extensions) define a rich ontology — 10 Noetics, 4 Domains, 7 Drives, 4 Registers, and the generative-operation notation governed by the v7.7 operator discipline — but the production notation compiler enforces value *kinds* (state, drive, relation, program) rather than categorical *source and target objects*. Three failure modes follow from that gap, each observed during the Project Coherence mathematical red-team audit:

1. **Untyped composition.** Nothing in a kind system prevents composing two operations whose intermediate objects disagree, silently producing an expression with no denotation.
2. **Invalid power inference.** A sequence of heterogeneous, time-varying generators is not an element of one finite semigroup; treating an arbitrary generator sequence as "one operator iterated n times" licenses false fixed-point and idempotence conclusions.
3. **Nondeterministic identity.** When carrier order is left to hash order, basis indices, denotations, and canonical representatives silently depend on interpreter state (`PYTHONHASHSEED`), which destroys replayability.

TKS v8.0 answers these with the smallest kernel that makes each failure a type error, while explicitly refusing to over-claim: the kernel is a semantic substrate that *future* TKS/CAS constructs may be mapped onto after review, not a formalization of the existing canon.

2. The typed finite-function category

2.1 Objects and arrows

An object is a pair `X = (name_X, carrier_X)` where `carrier_X` is a finite, explicitly ordered sequence of distinct hashable elements. Object identity includes the declared name as well as the carrier, preventing accidental coercion between semantically distinct objects of equal cardinality. All order-sensitive inputs (carrier elements, arrow mappings, path arrows, word labels, catalog generators, lift coordinate vectors) reject sets, mappings, and one-shot iterables: callers must provide a replayable ordered sequence, because order changes basis indices, denotations, and shortlex rank.

An arrow judgment `f : X -> Y` is a total map from every standard source index of `X` to one standard target index of `Y`. Extensional equality is equality of source, target, and the full mapping tuple; presentation names do not affect equality.

2.2 Identity and composition

For each object, `id_X(i) = i`. Given `f : X -> Y` and `g : Y -> Z`,

```
(g ∘ f)(i) = g(f(i)) : X -> Z,
```

and the implementation rejects `g ∘ f` unless the declared target of `f` *equals* the declared source of `g` — no cardinality-only or carrier-only coercion is performed. Left identity, right identity, and associativity follow pointwise for total finite functions; the executable suite exercises these laws exhaustively on a bounded family (19,683 three-point associativity triples), and a pinned Lean module (`TypedCategory`) kernel-checks the identity and associativity lemmas exactly.

2.3 Endomaps versus semigroupoid paths

The kernel's central discipline is the separation of two constructions that informal notation tends to conflate:

- **`Endomap(f)`** is constructible only when `f : X -> X`. It admits the monoid power `f^n`, including `f^0 = id_X`. The classical finite-semigroup result that `f^(|X|!)` is idempotent is implemented and exhaustively tested for carriers through size four: after `|X|!` iterations every point has entered its functional-graph cycle, and every cycle length at most `|X|` divides `|X|!`.
- **`SemigroupoidPath`** is a nonempty sequence `X0 -f1-> X1 -f2-> ... -fn-> Xn` whose adjacent object types match. It may cross carriers, has a typed composite, and *intentionally has no power operation*. This blocks the invalid inference from arbitrary typed or time-varying generator sequences to one finite semigroup element.

Exponentiation by squaring uses bounded internal presentation names (bit length plus a short SHA-256 label for large exponents), so presentation-name construction cannot hit Python's decimal-digit conversion limit; regression tests cover `2^30` and `10^5000`.

3. The deterministic standard-basis lift

For `f : X -> Y`, let `H_X` and `H_Y` be the finite-dimensional Hilbert spaces with orthonormal bases indexed by `X` and `Y`. Define the lift

```
A_f : H_X -> H_Y,     A_f e_x = e_{f(x)}.
```

3.1 Exact operator-norm theorem

For nonempty `X` (empty-domain value defined as zero):

```
||A_f||^2 = max_{y in Y} |f^{-1}(y)|.
```

Proof sketch. The `(x, x')` entry of `A_f* A_f` is 1 exactly when `f(x) = f(x')`. Grouping the input basis by fibers, this Gram matrix is a direct sum of all-ones blocks `J_m`, one per nonempty fiber; the eigenvalues of `J_m` are `m` (once) and `0` (multiplicity `m − 1`). The largest eigenvalue of `A_f* A_f` is therefore the maximum fiber size. ■

Implemented consequences:

```
tr(A_f* A_f) = |X|,        rank(A_f) = |im(f)|,
A_f is an isometry  iff  f is injective.
```

`A_f* A_f` is the input-collision Gram matrix; `A_f A_f*` is diagonal with fiber sizes; the codomain support projector is the separate diagonal 0/1 matrix marking nonempty fibers. These are deliberately distinct operations.

A permanent regression fixture records the constant map on two points:

```
A_f = [1 1]
      [0 0]
```

is idempotent yet has norm √2, trace-Gram 2, rank 1, and is neither a partial isometry nor an orthogonal projector. This fixture blocks the tempting but false general claims "deterministic lifts are contractive" and "idempotent lifts are projectors."

3.2 Functoriality

The lift satisfies the finite standard-basis functor laws

`A_(id_X) = I_(H_X),` `A_(g ∘ f) = A_g A_f,`

checked by exact matrix equality for every valid typed-map pair on carriers through size three (1,678 composition pairs, including empty carriers). The Lean seed additionally kernel-checks identity entrywise and on basis vectors, typed composition on basis vectors, and relational-product support composition. Finite-sum numerical matrix multiplication and arbitrary-vector linearity remain `OPEN`.

4. Stable denotational normal forms

Let `G = (g_0, ..., g_{k−1})` be a finite, explicitly ordered set of typed generators. A typed word carries a start object, and its labels must compose in sequence. Shortlex compares word length first, then the declared generator rank sequence. For a reachable finite-function denotation `a`:

`NF_G(a) = the shortlex-least well-typed word w with [[w]] = a.`

The closure algorithm explores words in shortlex order and expands only the first representative of each extensional denotation. The pruning is sound: if two prefixes share a typed denotation, appending the same well-typed suffix preserves both denotational equality and the shortlex advantage of the earlier prefix. The closure is finite (at most `|Y|^|X|` total maps per object pair over a finite object set); the implementation returns only after closure completion, deduplicates queued candidates by typed denotation, and raises `ClosureLimitExceeded` rather than ever returning a partial table as canonical.

Two boundaries are explicit. First, this is denotational minimization over a completed finite closure — **not** a claim that any inherited TKS rewrite system terminates or is confluent. Second, changing the declared generator order may change the selected representative without changing its denotation; canonical forms are relative to a declared, versioned generator order.

5. Formal verification status

A pinned, isolated Lean 4.32.1 toolchain (commit `f054605aea4b840552cca2e725580bffd1e1b704`) kernel-checks a named subset of the results. The verifier (`scripts/verify_psr8_lean_gate1.py`) hardcodes the sole allowed standard axiom (`propext`), rejects project axiom declarations and `sorry`, checks source hashes, builds through Lake, and audits exact theorem dependency reports.

Verified scope (Lean): typed total-function identities and associativity; endomap-only powers; the exact two-state constant-lift expansion and noncontractivity counterexample; deterministic-lift identity entrywise and on basis vectors; typed basis composition; relational-product support composition.

Open scope (Lean): the universal lift-norm theorem; the generic bridge from the two-state fixture to matrix/Gram/trace/rank definitions; finite-sum numerical matrix multiplication; arbitrary-vector

linear extension; the factorial idempotent-power theorem; shortlex leastness; any presheaf/sheaf or product-site formalization.

6. Proof and integration ledger

ID	Obligation	Status
PO-TKS8-CORE-01	Identity/associativity of finite-function composition	`PARTIAL FORMAL VERIFICATION PRESENT`
PO-TKS8-CORE-02	Reject every source/target mismatch	`EXECUTABLE_VALIDATION`
PO-TKS8-CORE-03	Keep one-carrier powers distinct from heterogeneous paths	`EXECUTABLE_VALIDATION`
PO-TKS8-CORE-04	`f^(|X|!)` idempotent for finite endomaps	`STANDARD_RESULT / FORMALIZATION OPEN`
PO-TKS8-CORE-05	Exact deterministic-lift norm formula	`INFORMAL_PROOF_PRESENT / UNIVERSAL FORMALIZATION OPEN`
PO-TKS8-CORE-06	Soundness and leastness of shortlex catalog	`INFORMAL_CORRECTNESS_ARGUMENT / FORMALIZATION OPEN`
PO-TKS8-CORE-07	Independent theorem-prover formalization	`PARTIAL / PRESENT`
PO-TKS8-CORE-08	Reconcile kernel types with corrected v7.4 theorem records	`OPEN / NOT LICENSED`
PO-TKS8-CORE-09	Integrate with the active notation compiler	`OPEN`
PO-TKS8-CORE-10	Independent specialist review	`OPEN`
PO-TKS8-CORE-11	Lift preserves identities and composition	`PARTIAL FORMAL VERIFICATION / FINITE-SUM MATRIX BRIDGE OPEN`

7. Relation to the wider TKS/CAS program

The kernel is the finite-arrow boundary object for three downstream candidates described in companion white papers:

- **Noetic Topos 2.0** builds its marked context-factor site and modal candidate over a legacy category whose future reconciliation with kernel objects is obligation PO-TKS8-CORE-08.
- **CAS PCBA 8** may adapt only a *certified* total, finite, deterministic, stateless component transition to a kernel `FiniteArrow`; chains become `SemigroupoidPath` values and feedback powers require a certified `Endomap`.
- **CAS Core 8** uses the kernel for legacy fractal-digit composition checks: an explicit caller-supplied catalog of typed arrows, with missing digits or source/target mismatches failing closed.

In each case the kernel supplies type-correct composition only. No adapter carries an automatic CAS semantic theorem; mapping the ten canonical Noetics or any CAS construct onto kernel arrows remains `Ring 4 / NOT LICENSED` pending review.

8. What TKS v8.0 does not claim

Excluded and still OPEN: parsing or evaluating TKS v7.4/v8 surface syntax; mapping canonical Noetics or CAS constructs to kernel arrows; RPM monad data or laws; a rewrite system for inherited TKS equations; Noetic Topos 2.0 coverage/modalities; stochastic kernels, quantum channels, or empirical dynamics; and any transfer from the Ten Advances manuscript. No empirical, cognitive-dynamics, channel-contractivity, or CAS-efficacy claim follows from any result above.

9. Reproduction

From the repository root with src on PYTHONPATH:

```
$env:PYTHONPATH=(Resolve-Path src)
python -m unittest discover -s tests/tks_v8_core -v
python scripts/verify_tks_v8_gate1.py
```

The 2026-08-04 validation snapshot records PASS on 20 tests with the bounded exhaustive case counts cited in the abstract. Passing establishes only the behavior covered by this finite executable model; it does not close the open formalization, integration, semantic-transfer, or review obligations.

10. Conclusion

TKS v8.0 contributes a disciplined foundation rather than a sweeping theory: typed composition that cannot silently mistype, powers that exist only where a monoid exists, a lift whose exact norm is proved rather than assumed, and canonical forms that are deterministic because their inputs are ordered. The kernel's refusal to encode the TKS/CAS ontology before review is deliberate — the mapping of canon onto substrate is itself a gated research obligation, and preserving that boundary is part of the result.

References

- Project Coherence, *Mathematical Red-Team Report* v0.1 (MATHEMATICAL_RED_TEAM_REPORT_v0.1.md).
- Project Coherence, *Theorem Ledger* v0.1, records THM-LIFT-NORM-FIBER-FORMULA, THM-LIFT-OCCUPATION-COUNTEREXAMPLE, THM-FINITE-IDEMPOTENT-POWER.
- LEAN_PROOF_MANIFEST_v0.1.json — exact scope of the pinned Lean 4.32.1 seed.
- S. Mac Lane, *Categories for the Working Mathematician* — background for the categorical laws exercised (standard result; no novelty claimed).

Publication boundary. This white paper is an internal research artifact. It is not a released specification, peer-reviewed paper, Evidence Packet, product claim, license decision, canon amendment, or public disclosure authorization. New matter relative to USPTO provisional 64/037,173 requires its own filing or inclusion in the utility conversion (deadline ~2027-04-13).